Numeric And Arithmetic Operations in R

Arithmetic operations in R are used to perform basic mathematical calculations on numbers, vectors, matrices, and other data objects. R provides a set of built-in arithmetic operators that make numerical computations simple and efficient. These operators are widely used in statistical analysis, data manipulation, scientific computing, and programming. Arithmetic operations can be performed on individual values as well as on vectors, where R automatically performs element-wise calculations.

The basic arithmetic operators in R are:

Examples:

Addition:

```
a <- 10
b <- 5
a + b
# Output: 15
```

Subtraction:

```
a - b
# Output: 5
```

Multiplication:

```
a * b
# Output: 50
```

Division:

```
a / b
# Output: 2
```

Exponentiation:

```
2 ^ 4
# Output: 16
```

Modulus:

```
10 %% 3
# Output: 1
```

Integer Division:

```
10 %/% 3
# Output: 3
```

Arithmetic operations can also be performed on vectors. R applies the operation to each corresponding element of the vectors.

```
x <- c(2, 4, 6)
y <- c(1, 2, 3)
x + y
# Output: 3 6 9

x * y
# Output: 2 8 18
```

Arithmetic operations in R have many practical applications. They are used for statistical calculations, financial analysis, scientific computations, data transformation, machine learning, and creating mathematical models. These operations help programmers perform calculations efficiently while writing concise and readable code.

Assigning Objects in R

Assigning objects in R is the process of storing data such as numbers, text, vectors, matrices, data frames, or functions in memory using a variable name. These variables are known as objects because R treats almost everything as an object. Object assignment allows users to store, retrieve, modify, and reuse data efficiently throughout a program. It is one of the basic concepts of R programming and is essential for performing data analysis and statistical computations.

SBC KARKALA

In R, objects are assigned using the assignment operator <-, which is the most commonly used method. R also supports the = operator for assignment, and the -> operator can be used to assign a value to an object from left to right. Among these, the <- operator is the recommended and most widely used style in R programming.
The general syntax for assigning an object is:
object_name <- value
Here, object_name is the name of the variable, and value is the data assigned to it.
Examples:
Assign a numeric value:

```
x <- 100
x
# Output: 100
```

Assign a character string:

```
name <- "R Programming"
name
# Output: "R Programming"
```

Assign a vector:

```
marks <- c(75, 82, 90, 88)
marks
# Output: 75 82 90 88
```

Assign using the = operator:

```
a = 50
a
# Output: 50
```

Assign using the -> operator:

```
25 -> b
b
# Output: 25
```

Object names in R should begin with a letter or a dot (.) not followed by a number. They can contain letters, numbers, dots, and underscores but should not include spaces or special characters. Meaningful object names, such as student_marks or total_sales, make programs easier to read and maintain.
Assigning objects has many practical applications. It is used to store input data, save calculation results, create datasets, define functions, and manage variables during program execution. Object assignment also allows the same data to be reused in multiple calculations without re-entering values, improving the efficiency and readability of R programs.

Vectors in R

A vector is the most basic and important data structure in R. It is a one-dimensional collection of elements that are all of the same data type, such as numeric, character, logical, integer, or complex. Vectors are used to store and manipulate multiple values under a single variable name, making data processing and analysis efficient. Since R is a vector-oriented programming language, many operations are performed directly on vectors without the need for explicit loops.
Vectors are commonly created using the c() (combine or concatenate) function, which combines multiple values into a single vector.
The general syntax for creating a vector is:
vector_name <- c(value1, value2, value3, ...)
Here, vector_name is the name of the vector, and the values inside c() become the elements of the vector.
Examples:
Create a numeric vector:

```
numbers <- c(10, 20, 30, 40, 50)
numbers
```

SBCKARKALA

```
# Output: 10 20 30 40 50
Create a character vector:
names <- c("Anita", "Ravi", "Kiran", "Sita")
names
# Output: "Anita" "Ravi" "Kiran" "Sita"
Create a logical vector:
status <- c(TRUE, FALSE, TRUE)
status
# Output: TRUE FALSE TRUE
Access vector elements:
numbers[3]
# Output: 30
Perform arithmetic operations on vectors:
x <- c(2, 4, 6)
y <- c(1, 2, 3)
x + y
# Output: 3 6 9
```

Vectors have many practical applications in R. They are used to store datasets, perform mathematical and statistical calculations, create sequences of values, represent observations, and prepare data for graphs and machine learning models. Because R performs element-wise operations on vectors, data analysis becomes faster and more efficient.

Useful vector functions include:

- length() – Returns the number of elements in a vector.
- sort() – Sorts the elements in ascending or descending order.
- seq() – Generates a sequence of values.
- rep() – Repeats elements or vectors multiple times.

seq() Function in R

The seq() function in R is a built-in function used to generate sequences of numbers or dates. It is one of the most commonly used functions in R programming because it helps create ordered values automatically instead of entering them manually. The function is useful in statistical analysis, data manipulation, plotting graphs, simulations, and writing loops. By using seq(), users can generate sequences with a specified starting value, ending value, step size, or a fixed number of elements, making programming more efficient and reducing errors.

The general syntax of the seq() function is:

```
seq(from, to, by, length.out)
```

The from argument specifies the starting value of the sequence, while to specifies the ending value. The by argument determines the increment or decrement between consecutive values. Instead of specifying the step size, the length.out argument can be used to generate a sequence containing a fixed number of equally spaced values. Depending on the requirement, either by or length.out is generally used.

Examples:

```
Generate numbers from 1 to 10:
seq(1, 10)
# Output: 1 2 3 4 5 6 7 8 9 10
Generate even numbers from 2 to 20:
seq(2, 20, by = 2)
# Output: 2 4 6 8 10 12 14 16 18 20
Generate a descending sequence:
seq(10, 1, by = -1)
# Output: 10 9 8 7 6 5 4 3 2 1
```

Generate five equally spaced numbers between 1 and 10:
seq(1, 10, length.out = 5)
Output: 1.00 3.25 5.50 7.75 10.00
The seq() function has many practical applications. It is used to generate index values in for loops, create sequences for statistical computations, produce values for plotting graphs, generate time and date sequences, and prepare datasets for simulations and mathematical operations. It also improves code readability and minimizes manual data entry.

Repetition with rep() in R
The rep() function in R is a built-in function used to repeat values, vectors, or sequences multiple times. It is one of the most useful functions for creating repeated data efficiently without manually entering the same values. The rep() function is widely used in data analysis, statistical programming, simulations, and creating sample datasets. It can repeat each element individually, repeat the entire vector several times, or generate a vector of a specified length.
The general syntax of the rep() function is:
rep(x, times, each, length.out)
Here, x is the object or vector to be repeated. The times argument specifies how many times the entire vector should be repeated. The each argument repeats each element of the vector individually before moving to the next element. The length.out argument specifies the total length of the output vector, even if the repetitions are incomplete.
Examples:
Repeat a single value five times:
rep(5, times = 5)
Output: 5 5 5 5 5
Repeat an entire vector three times:
rep(c(1, 2, 3), times = 3)
Output: 1 2 3 1 2 3 1 2 3
Repeat each element twice:
rep(c(1, 2, 3), each = 2)
Output: 1 1 2 2 3 3
Repeat a vector to obtain a specific length:
rep(c("A", "B"), length.out = 5)
Output: "A" "B" "A" "B" "A"
The rep() function has many practical applications. It is used to create repeated patterns in datasets, generate sample data for testing programs, prepare labels for statistical analysis, repeat factor levels, and build vectors for simulations and experiments. It also helps reduce coding effort and improves program readability by avoiding manual repetition.

Sorting with sort() in R
The sort() function in R is a built-in function used to arrange the elements of a vector in ascending or descending order. It is one of the most commonly used functions in data analysis because it helps organize data, making it easier to understand, compare, and analyze. The sort() function works with numeric, character, and logical vectors. It is widely used in statistical analysis, report generation, and data preprocessing before performing further calculations.
The general syntax of the sort() function is:
sort(x, decreasing = FALSE, na.last = TRUE)
Here, x represents the vector to be sorted. The decreasing argument determines the sorting order. By default (FALSE), the elements are arranged in ascending order. If decreasing = TRUE, the elements are arranged in

SBC KARKALA

descending order. The na.last argument specifies whether missing values (NA) should appear at the end, beginning, or be removed from the sorted result.
Examples:
Sort a numeric vector in ascending order:
```
x <- c(8, 3, 5, 1, 9)
sort(x)
# Output: 1 3 5 8 9
```
Sort a numeric vector in descending order:
```
sort(x, decreasing = TRUE)
# Output: 9 8 5 3 1
```
Sort a character vector alphabetically:
```
names <- c("Ravi", "Anita", "Kiran", "Balu")
sort(names)
# Output: "Anita" "Balu" "Kiran" "Ravi"
```
Sort a vector containing missing values:
```
x <- c(5, NA, 2, 8, 1)
sort(x, na.last = TRUE)
# Output: 1 2 5 8 NA
```
The sort() function has several practical applications. It is used to arrange data before analysis, rank values, organize names alphabetically, prepare datasets for reporting, and simplify the identification of minimum and maximum values. Sorting also improves data visualization and makes statistical computations more effective.

Finding a Vector Length with length() in R

The length() function in R is a built-in function used to determine the number of elements in a vector or other R objects. It is one of the most fundamental functions in R programming because it helps users identify the size of a dataset or vector before performing operations. The length() function is widely used in data analysis, loops, conditional statements, and statistical computations. It returns a single integer value representing the total number of elements present in the object.
The general syntax of the length() function is:
```
length(x)
```
Here, x represents the vector or object whose length is to be determined. The function counts all elements in the vector, including repeated values and missing values (NA). It does not count the number of unique values; it simply returns the total number of elements.
Examples:
Find the length of a numeric vector:
```
x <- c(10, 20, 30, 40, 50)
length(x)
# Output: 5
```
Find the length of a character vector:
```
names <- c("Anita", "Ravi", "Kiran", "Sita")
length(names)
# Output: 4
```
Find the length of a vector containing missing values:
```
x <- c(5, 8, NA, 12, 15)
length(x)
# Output: 5
```
Find the length of an empty vector:
```
x <- c()
length(x)
# Output: 0
```
SBC KARIKALA

The length() function has many practical applications. It is used to determine the size of vectors before processing data, control the number of iterations in loops, validate input data, compare the sizes of different vectors, and check whether a vector is empty. It is also useful in writing efficient R programs by ensuring that operations are performed on vectors of the correct size.

Subsetting and Element Extraction in R

Subsetting is the process of selecting one or more elements from a vector, matrix, list, or data frame in R. It is one of the most important operations in R programming because it allows users to access, extract, and manipulate specific parts of a dataset. Every element in a vector has a unique index (position), and R uses these indexes to retrieve the required values. Indexing in R starts from 1, which means the first element has index 1, the second element has index 2, and so on.

When a vector is printed in the R console, numbers such as [1], [6], or [11] may appear on the left side of the output. These numbers indicate the index of the first element displayed on that line. They are not part of the vector's data but help users identify the position of elements within the vector.

The general syntax for extracting elements from a vector is:

vector_name[index]

Here, vector_name is the name of the vector, and index specifies the position of the element to be extracted.

Examples:

Create a vector:

```
myvec <- c(10, 20, 30, 40, 50)
```

Extract the third element:

```
myvec[3]
# Output: 30
```

Extract multiple elements:

```
myvec[c(2, 4, 5)]
# Output: 20 40 50
```

Extract a range of elements:

```
myvec[2:4]
# Output: 20 30 40
```

Exclude an element:

```
myvec[-3]
# Output: 10 20 40 50
```

Extract elements using logical conditions:

```
myvec[myvec > 25]
# Output: 30 40 50
```

Subsetting has many practical applications in R. It is used to retrieve specific values from datasets, filter observations based on conditions, remove unwanted elements, perform calculations on selected data, and prepare data for statistical analysis and visualization. It is also useful when working with large datasets, as only the required elements need to be processed.

Vector-Oriented Behavior in R

Vector-oriented behavior (also called vectorized behavior) is an important feature of R that allows operations to be performed on all elements of a vector at the same time. Instead of using loops to process each value individually, R automatically applies the operation element by element. This makes calculations faster, simpler, and more efficient.

In R, when two vectors are used in an arithmetic operation, the elements are matched according to their positions. The first element of one vector operates with the first element of another vector, the second with the second, and so on. The result is returned as a new vector.

Simple Example:

SBC KARKALA

```
x <- c(10, 20, 30)
y <- c(1, 2, 3)
x + y
```

Output:

```
[1] 11 22 33
```

In this example, R performs addition element by element:

10 + 1 = 11

20 + 2 = 22

30 + 3 = 33

Thus, R completes the entire calculation in a single statement without requiring a loop.

Vector-oriented behavior also supports operations between vectors of different lengths through a process called recycling. In recycling, the shorter vector is repeated until it matches the length of the longer vector.

Example:

```
x <- c(10, 20, 30, 40)
y <- c(2, 5)
x + y
```

Output:

```
[1] 12 25 32 45
```

Here, the vector y is recycled as 2, 5, 2, 5 and added to the elements of x.

Advantages of Vector-Oriented Behavior:

Reduces the need for loops.

Makes programs shorter and easier to understand.

Improves execution speed.

Useful for statistical analysis and data manipulation.

Defining a Matrix in R

A matrix in R is a two-dimensional data structure that stores elements of the same data type in rows and columns. It is similar to a table or a mathematical matrix, where each element is identified by its row and column position. Matrices are commonly used in mathematical calculations, statistical analysis, linear algebra, and data processing.

In R, a matrix is created using the matrix() function. The function allows users to specify the values, number of rows, and number of columns of the matrix.

The general syntax for creating a matrix is:

```
matrix(data, nrow, ncol, byrow = FALSE)
```

Arguments:

data – The values that will be stored in the matrix.

nrow – The number of rows in the matrix.

ncol – The number of columns in the matrix.

byrow – Determines how values are filled. By default, R fills the matrix column-wise. If byrow = TRUE, values are filled row-wise.

Example 1: Creating a matrix by columns

```
m <- matrix(data=c(1, 2, 3, 4, 5, 6), nrow = 2, ncol = 3)
m
```

Output:

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

Example 2: Creating a matrix by rows

```
m <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2, ncol = 3,byrow = TRUE)
```


SBCKARKALA

m
Output:
```
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```
Elements of a matrix can be accessed using row and column indexes. The syntax for accessing an element is:
matrix_name[row, column]
Example:
m[1,2]
Output:
[1] 2
Matrices support various operations such as addition, subtraction, multiplication, transpose, and finding dimensions. Some useful matrix functions in R include:
dim() – Returns the dimensions of a matrix.
nrow() – Gives the number of rows.
ncol() – Gives the number of columns.
t() – Finds the transpose of a matrix.
Matrices are widely used in applications such as statistical modeling, machine learning, image processing, and numerical computations because they provide an efficient way to store and manipulate structured data.

Filling Direction in R Matrices
In R, when a matrix is created using the matrix() function, the values are arranged in a specific direction. By default, R fills the matrix column-wise, meaning that the elements are placed from top to bottom in the first column, then the second column, and so on. This arrangement is known as the filling direction of a matrix.
The general syntax for creating a matrix is:
matrix(data, nrow, ncol, byrow = FALSE)
The byrow argument controls the direction in which values are filled:
byrow = FALSE (default) fills the matrix column-wise.
byrow = TRUE fills the matrix row-wise.
1. Column-wise Filling (Default)
When byrow is not specified or set to FALSE, R fills the matrix column by column.
Example:
m <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2)
m
Output:
```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```
Here, R places 1 and 2 in the first column, 3 and 4 in the second column, and 5 and 6 in the third column.
2. Row-wise Filling
To fill a matrix row by row, the argument byrow = TRUE is used.
Example:
m <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2, byrow = TRUE)
m
Output:

```
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```


SBC KARKALA

In this case, R fills the first row with 1, 2, 3 and the second row with 4, 5, 6.

Importance of Filling Direction

The filling direction is important because it determines how data is organized inside a matrix. Choosing the correct filling direction helps maintain the intended structure of data and avoids incorrect calculations during matrix operations. It is especially useful when working with tables, datasets, and mathematical matrices.

Row and Column Bindings in R

In R, row binding and column binding are methods used to quickly create a matrix by combining multiple vectors of equal length. R provides two built-in functions, rbind() and cbind(), for combining vectors into a matrix. These functions allow vectors to be arranged either as rows or as columns.

Row Binding using rbind()

The rbind() function is used to bind vectors together as rows of a matrix. Each vector supplied to rbind() becomes a separate row in the resulting matrix. The order of the rows depends on the order in which the vectors are provided.

Syntax:

rbind(vector1, vector2, ...)

Example:

rbind(1:3, 4:6)

Output:

```
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```

In this example, the two vectors 1:3 and 4:6 are combined as rows. The vector 1:3 becomes the first row, and 4:6 becomes the second row. The result is a 2 × 3 matrix containing two rows and three columns.

Column Binding using cbind()

The cbind() function is used to bind vectors together as columns of a matrix. Each vector supplied to cbind() becomes a separate column in the resulting matrix. The order of the columns follows the order of the vectors given in the function.

Syntax:

cbind(vector1, vector2, ...)

Example:

cbind(c(1,4), c(2,5), c(3,6))

Output:

```
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```

In this example, three vectors of length 2 are combined using cbind(). Each vector becomes a column of the matrix. The first vector c(1,4) forms the first column, c(2,5) forms the second column, and c(3,6) forms the third column.

Applications

Row and column binding functions are useful for constructing matrices from existing vectors. They are commonly used in data analysis, statistical computing, and data organization. These functions save time by allowing users to create matrices without manually entering each element.

Matrix Dimensions in R

A matrix dimension represents the size of a matrix in terms of the number of rows and columns it contains. In R, every matrix has two dimensions: the number of rows and the number of columns. The dimensions of a matrix determine its structure and help in performing various matrix operations.

A matrix with m rows and n columns is called an m × n matrix. For example, a matrix with 3 rows and 2 columns is called a 3 × 2 matrix.

SBC KARKALA

In R, the dimensions of a matrix can be found using the built-in function dim(). The dim() function returns the number of rows and columns of a matrix.
Syntax:
dim(matrix_name)
Example:
m <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2,nco=3)
m
Output:
```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

Finding the dimensions:
dim(m)
Output:
[1] 2 3
The output shows that the matrix has 2 rows and 3 columns, so it is a 2 × 3 matrix.
Finding Rows and Columns Separately
R also provides two functions to find the number of rows and columns individually:
1. nrow() function
The nrow() function returns the number of rows in a matrix.
Example:
nrow(m)
Output:
[1] 2
2. ncol() function
The ncol() function returns the number of columns in a matrix.
Example:
ncol(m)
Output:
[1] 3

Matrix dimensions are important because they determine whether mathematical operations can be performed between matrices. For example, matrix addition requires matrices to have the same dimensions, while matrix multiplication depends on the relationship between rows and columns.
Matrix dimensions are widely used in data analysis, statistics, machine learning, and linear algebra to organize and process structured data efficiently.

Matrix Subsetting in R
Matrix subsetting is the process of extracting specific elements, rows, columns, or parts of a matrix in R. Since a matrix contains data arranged in rows and columns, elements can be accessed by specifying their **row index and column index**. Matrix subsetting is an important operation in R because it allows users to select and manipulate required portions of data without using the complete matrix.
In R, matrix elements are accessed using square brackets [ ]. The general syntax for extracting elements from a matrix is:
matrix_name[row_number, column_number]
Here:
- row_number specifies the row position.
- column_number specifies the column position.

CBC KARKALA

Example:

Create a matrix:

```
m <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2)
m
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

Extracting a Single Element

To access a specific element, specify both row and column indexes.

Example:

```
m[1,3]
```

output

```
[1] 5
```

Row, Column, and Diagonal Extractions in R Matrix

In R, a matrix is a two-dimensional data structure consisting of rows and columns. Row, column, and diagonal extractions are important operations used to retrieve specific parts of a matrix. These operations allow users to access required data efficiently for analysis and calculations. R provides simple indexing methods and built-in functions to extract rows, columns, and diagonal elements from a matrix.

1. Row Extraction

A complete row of a matrix can be extracted by specifying the row number and leaving the column position blank.

The general syntax is:

```
matrix_name[row_number, ]
```

Example:

```
m <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2)
m
```

Output:

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

Extract the second row:

```
m[2, ]
```

Output:

```
[1] 2 4 6
```

The second row containing all its elements is returned.

2. Column Extraction

A complete column of a matrix can be extracted by specifying the column number and leaving the row position blank.

Syntax:

```
matrix_name[, column_number]
```

Example:

Extract the third column:

```
m[,3]
```

Output:

```
[1] 5 6
```

The third column containing all its elements is returned.

3. Diagonal Extraction

SBC KARKALA

The diagonal elements of a matrix are the elements that occur at the same row and column positions, such as the elements from the top-left corner to the bottom-right corner. In R, the built-in function diag() is used to extract diagonal elements.
Syntax:
diag(matrix_name)
Example:
Create a square matrix:
m <- matrix(c(1, 2, 3, 4, 5, 6, 7, 8, 9), nrow = 3)
m
Output:
```
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
```
Extract diagonal elements:
diag(m)
Output:
[1] 1 5 9
The diagonal elements 1, 5, and 9 are extracted from the matrix.

Row, column, and diagonal extractions are widely used in data analysis, statistical computations, and matrix operations. They help in selecting specific data, performing calculations on parts of a matrix, and analyzing patterns within structured datasets.

Omitting and Overwriting in R Matrix
In R, elements omitting and overwriting are important techniques used for modifying matrices. Omitting allows users to remove specific elements, rows, or columns from a matrix, while overwriting allows users to replace existing values with new values. These operations are useful for data cleaning, correction, and manipulation during data analysis.

1. Omitting Elements from a Matrix
In R, elements, rows, or columns can be omitted by using the negative indexing operator (-). Negative indexes indicate the elements that should be removed from the matrix.
Example: Omitting a Row
Create a matrix:
m <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2)
m
Output:
```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```
Remove the first row:
m[-1,]
Output:
[1] 2 4 6
The first row is omitted, and only the remaining row is displayed.
Example: Omitting a Column
m[, -2]
Output:

SBC KARKALA

```
     [,1] [,2]
[1,]    1    5
[2,]    2    6
```

The second column is removed from the matrix.

2. Overwriting Elements in a Matrix

Overwriting means replacing existing values in a matrix with new values. In R, values can be changed by assigning a new value to a specific row and column position.

Syntax:

```
matrix_name[row, column] <- new_value
```

Example:

```
m <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2)
m[1,2] <- 10
m
```

Output:

```
     [,1] [,2] [,3]
[1,]    1   10    5
[2,]    2    4    6
```

In this example, the original value at the position row 1, column 2 is replaced with 10.

Omitting and overwriting are commonly used in data manipulation tasks. They help in removing unwanted data, correcting incorrect values, updating datasets, and preparing matrices for statistical analysis. These operations are especially useful when working with large datasets where manual modification is difficult.

Matrix Operations and Algebra

In R, matrices can be understood in two different ways. First, matrices can be used as a **data storage structure** for storing and managing values during programming. In this case, matrices are mainly used to organize data in rows and columns and perform basic operations on them.

The behavior of matrices in mathematical calculations is different from their general data handling behavior in R. For example, normal multiplication of matrix elements is different from mathematical matrix multiplication. Therefore, it is important to understand the purpose for which a matrix is being used.

R provides several built-in functions and operators to perform mathematical operations on matrices.

These include matrix addition, subtraction, multiplication, transpose, and finding special matrices such as identity matrices.

Matrix Transpose in R

The transpose of a matrix is an operation that converts the rows of a matrix into columns and the columns into rows. In other words, the elements of a matrix are rearranged by changing their positions across the main diagonal. The transpose of a matrix is represented by the symbol A^T.

In R, the transpose of a matrix is obtained using the built-in function t(). This function returns a new matrix where rows and columns are interchanged.

Syntax:

```
t(matrix_name)
```

Here, matrix_name represents the matrix whose transpose is required.

Example:

Create a matrix:

```
m <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2)
m
```

Output:

```
     [,1] [,2] [,3]
```

[1,] 1 3 5
[2,] 2 4 6
Find the transpose of the matrix:
t(m)
Output:
[,1] [,2]
[1,] 1 2
[2,] 3 4
[3,] 5 6
In this example, the original matrix has 2 rows and 3 columns. After applying the t() function, the resulting matrix has 3 rows and 2 columns. The rows and columns have been interchanged.

Properties of Matrix Transpose:

The transpose of a matrix changes rows into columns and columns into rows.
The dimensions of an m × n matrix become n × m after transposition.
The transpose of a transpose returns the original matrix:
$(A^T)^T = A$
The transpose of a square matrix may have the same dimensions as the original matrix.

Matrix transpose is used in many areas such as linear algebra, statistics, machine learning, and data analysis. It is useful for rearranging data, performing matrix multiplication, solving mathematical equations, and handling datasets stored in matrix form.

Identity Matrix in R

An identity matrix is a special type of square matrix in which all the elements on the main diagonal are equal to 1 and all other elements are 0. It is represented by the symbol I. An identity matrix acts like the number 1 in normal multiplication. When any matrix is multiplied by an identity matrix of the appropriate size, the original matrix remains unchanged.
An identity matrix is also called a unit matrix. In R, an identity matrix can be created using the diag() function.
Syntax:
diag(n)
Here, n specifies the size of the square identity matrix (number of rows and columns).
Example:
Create a 3 × 3 identity matrix:
I <- diag(3)
I
Output:
[,1] [,2] [,3]
[1,] 1 0 0
[2,] 0 1 0
[3,] 0 0 1
In this example, diag(3) creates a square matrix with 3 rows and 3 columns. The diagonal elements are 1, and all remaining elements are 0.

Properties of Identity Matrix:

It is always a square matrix.
All diagonal elements are 1.
All non-diagonal elements are 0.
The identity matrix does not change a matrix during multiplication.
The transpose of an identity matrix is the same identity matrix.

SBC KARIKALA

Identity matrices are used in linear algebra, matrix equations, transformations, computer graphics, statistics, and machine learning. They are important in solving matrix operations and representing neutral elements in matrix multiplication.

Scalar Multiple of a Matrix in R

A scalar multiple of a matrix is an operation in which every element of a matrix is multiplied by a single constant value called a scalar. A scalar is a single number, and when it is multiplied by a matrix, it is distributed to each element of the matrix. The resulting matrix has the same dimensions as the original matrix.

In R, scalar multiplication can be performed using the normal multiplication operator *. R automatically applies the multiplication to each element of the matrix.

Syntax:

```
scalar * matrix_name
or
matrix_name * scalar
```

Example:

Create a matrix:

```
A <- matrix(c(1, 2, 3, 4), nrow = 2,ncol=2)
A
```

Output:

```
     [,1] [,2]
[1,]    1    3
[2,]    2    4
```

Multiply the matrix by a scalar value 5:

```
5 * A
```

Output:

```
     [,1] [,2]
[1,]    5   15
[2,]   10   20
```

In this example, each element of matrix A is multiplied by 5:

1 × 5 = 5
2 × 5 = 10
3 × 5 = 15
4 × 5 = 20

The dimensions of the matrix remain unchanged after scalar multiplication.

Properties of Scalar Multiplication:

Every element of the matrix is multiplied by the same scalar value.
The size (rows and columns) of the matrix remains the same.
Multiplication by scalar zero produces a zero matrix.
Multiplication by scalar one gives the original matrix.
Scalar multiplication follows the distributive property.

Applications of Scalar Multiplication:

Scalar multiplication is used in mathematical calculations, data scaling, statistical analysis, image processing, and scientific computations. It is commonly used to increase or decrease the magnitude of data values while maintaining the same structure.

Matrix Addition and Subtraction in R

SBC KARKALA

Matrix addition and subtraction are basic mathematical operations performed on matrices. In R, these operations can be performed using the operators + for addition and - for subtraction. For matrix addition or subtraction, the matrices must have the same number of rows and columns. The operations are performed element by element, meaning that corresponding elements from the two matrices are added or subtracted.

1. Matrix Addition

Matrix addition is the process of adding two matrices by adding their corresponding elements. The resulting matrix has the same dimensions as the original matrices.

Syntax:

Matrix_A + Matrix_B

Example:

Create two matrices:

```
A <- matrix(c(1, 2, 3, 4), nrow = 2)
B <- matrix(c(5, 6, 7, 8), nrow = 2)
A + B
```

Output:

```
     [,1] [,2]
[1,]    6   10
[2,]    8   12
```

The addition is performed as:

1 + 5 = 6
2 + 6 = 8
3 + 7 = 10
4 + 8 = 12

2. Matrix Subtraction

Matrix subtraction is the process of subtracting corresponding elements of one matrix from another matrix. The resulting matrix also has the same dimensions as the original matrices.

Syntax:

Matrix_A - Matrix_B

Example:

```
A - B
```

Output:

```
     [,1] [,2]
[1,]   -4   -4
[2,]   -4   -4
```

The subtraction is performed as:

1 - 5 = -4
2 - 6 = -4
3 - 7 = -4
4 - 8 = -4

Conditions for Matrix Addition and Subtraction:

Both matrices must have the same dimensions.
Corresponding elements are added or subtracted.
The resulting matrix has the same size as the original matrices.
R performs these operations element-wise.

Applications of Matrix Addition and Subtraction:

Matrix addition and subtraction are used in statistical calculations, data manipulation, scientific computations, and mathematical modeling. They are useful for comparing datasets, updating matrices, and performing operations in linear algebra.

SBC KARKALA

Matrix Multiplication in R

Matrix multiplication is a mathematical operation in which two matrices are multiplied together to produce a new matrix. Unlike matrix addition and subtraction, matrix multiplication does not operate element by element. Instead, it involves multiplying the elements of rows in the first matrix with the elements of columns in the second matrix and adding the results.

In R, matrix multiplication is performed using the %*% operator. The number of columns in the first matrix must be equal to the number of rows in the second matrix for multiplication to be possible.

Syntax:

Matrix_A %*% Matrix_B

Example:

Create two matrices:

```
A <- matrix(c(1, 2, 3, 4), nrow = 2)
B <- matrix(c(5, 6, 7, 8), nrow = 2)
A
B
```

Matrix A:

```
     [,1] [,2]
[1,]    1    3
[2,]    2    4
```

Matrix B:

```
     [,1] [,2]
[1,]    5    7
[2,]    6    8
```

Perform matrix multiplication:

```
A %*% B
```

Output:

```
     [,1] [,2]
[1,]   23   31
[2,]   34   46
```

The calculation is performed as:

First element: (1 × 5) + (3 × 6) = 23

Second element: (1 × 7) + (3 × 8) = 31

Third element: (2 × 5) + (4 × 6) = 34

Fourth element: (2 × 7) + (4 × 8) = 46

Difference Between * and %*% in R:

* operator performs element-wise multiplication.

Example:

```
A * B
```

Each corresponding element is multiplied separately.

%*% operator performs mathematical matrix multiplication.

Example:

```
A %*% B
```

Rows and columns are used according to matrix multiplication rules.

Properties of Matrix Multiplication:

The order of multiplication is important: generally, A × B is not equal to B × A.

The number of columns of the first matrix must equal the number of rows of the second matrix.

The resulting matrix has dimensions based on the rows of the first matrix and columns of the second matrix.

SBC KARKALA

Matrix multiplication is widely used in linear algebra and statistical models.

Applications of Matrix Multiplication:

Matrix multiplication is used in regression analysis, machine learning algorithms, computer graphics, scientific calculations, and solving mathematical equations. It is an important operation for representing relationships between variables and performing complex computations.

Matrix Inversion in R

Matrix inversion is a mathematical operation used to find the inverse of a square matrix. The inverse of a matrix A is represented as A^{-1}. When a matrix is multiplied by its inverse, the result is an identity matrix.

$A \times A^{-1} = I$

where I represents the identity matrix.

A matrix can have an inverse only if it is a square matrix (same number of rows and columns) and its determinant is not equal to zero. A matrix without an inverse is called a singular matrix.

In R, matrix inversion is performed using the solve() function.

Syntax:

```
solve(matrix_name)
```

Example:

Create a square matrix:

```
A <- matrix(c(1, 2, 3, 4), nrow = 2)
A
```

Output:

```
     [,1] [,2]
[1,]    1    3
[2,]    2    4
```

Find the inverse of the matrix:

```
solve(A)
```

Output:

```
     [,1] [,2]
[1,]   -2    1.5
[2,]    1   -0.5
```

The resulting matrix is the inverse of matrix A.

Verification of Matrix Inversion:

The inverse can be checked by multiplying the original matrix with its inverse:

```
A %*% solve(A)
```

Output:

```
     [,1] [,2]
[1,]    1    0
[2,]    0    1
```

The result is an identity matrix, which confirms that the inverse calculation is correct.

Properties of Matrix Inversion:

Only square matrices can have inverses.

The determinant of the matrix must not be zero.

The inverse of an identity matrix is the identity matrix itself.

The inverse of a product of matrices follows the rule:

$(AB)^{-1} = B^{-1}A^{-1}$

A matrix multiplied by its inverse gives an identity matrix.

Applications of Matrix Inversion:

SBC KARKALA

Matrix inversion is used in solving systems of linear equations, regression analysis, statistics, machine learning, and scientific computations. It is an important tool in linear algebra for finding unknown values and performing mathematical transformations.

Multidimensional Arrays in R

A multidimensional array in R is a data structure that can store elements in more than two dimensions. While a matrix stores data in rows and columns (two dimensions), an array can store data in multiple dimensions such as rows, columns, and additional layers. Multidimensional arrays are useful for representing complex data structures in statistics, scientific computing, and data analysis.

An array in R contains elements of the same data type and is created using the built-in array() function. The dimensions of the array are specified using the dim argument.

Syntax:

array(data, dim = dimensions)

Arguments:

data – The values to be stored in the array.

dim – Specifies the size of each dimension of the array.

Example:

Create a 3-dimensional array:

```
arr <- array(c(1, 2, 3, 4, 5, 6, 7, 8),dim = c(2, 2, 2))
arr
```

Output:

```
, , 1
     [,1] [,2]
[1,]    1    3
[2,]    2    4
, , 2
     [,1] [,2]
[1,]    5    7
[2,]    6    8
```

In this example, the array has dimensions 2 × 2 × 2, meaning it contains:

2 rows

2 columns

2 layers (or matrices)

The values are arranged across different dimensions automatically by R.

Accessing Elements of a Multidimensional Array

Elements of an array can be accessed by specifying the index position of each dimension.

Syntax:

array_name[row, column, dimension]

Example:

```
arr[1, 2, 1]
```

Output:

```
[1] 3
```

This extracts the element located at the first row, second column, and first layer.

Finding Array Dimensions

The dimensions of an array can be found using the dim() function.

Example:

```
dim(arr)
```

SBC KARKALA

Output:
[1] 2 2 2
This indicates that the array has three dimensions.

For the R command:
BR <- array(data = rep(1:24, times = 3), dim = c(3, 4, 2, 3))
BR
Output:

```
, , 1, 1
     [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12
, , 2, 1
     [,1] [,2] [,3] [,4]
[1,]   13   16   19   22
[2,]   14   17   20   23
[3,]   15   18   21   24
, , 1, 2
     [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12
, , 2, 2
     [,1] [,2] [,3] [,4]
[1,]   13   16   19   22
[2,]   14   17   20   23
[3,]   15   18   21   24
, , 1, 3
     [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12
, , 2, 3
     [,1] [,2] [,3] [,4]
[1,]   13   16   19   22
[2,]   14   17   20   23
[3,]   15   18   21   24
```

Explanation:
The array dimension is c(3,4,2,3), so it contains:
3 rows
4 columns
2 layers
3 groups
Total elements:
3×4×2×3=72
The values 1:24 are repeated three times using rep(1:24, times=3).

SBC KARKALA

R fills the array column-wise, so the first 24 values fill the first two layers of the first group, and the same pattern repeats for the next two groups.

Subsets, Extractions, and Replacements in R

In R, subsetting, extraction, and replacement are important operations used to access, select, and modify elements of data structures such as vectors, matrices, and arrays. These operations allow users to work with specific parts of data instead of processing the entire structure.

1. Subsetting in R

Subsetting refers to selecting a specific part of a data structure using index values. In R, square brackets [ ] are used to access subsets of vectors, matrices, and arrays.

For a multidimensional array, the general syntax is:

array_name[row, column, dimension]

Example:

Create an array:

```
BR <- array(data = rep(1:24, times = 3), dim = c(3,4,2,3))
```

Extract a subset:

```
BR[1:2, 1:2, 1, 1]
```

Output:

```
     [,1] [,2]
[1,]    1    4
[2,]    2    5
```

This extracts the first two rows and first two columns from the first layer and first group of the array.

2. Extractions in R

Extraction means retrieving specific elements from a data structure by using their position or index values.

Example:

Extract a single element:

```
BR[2,3,1,1]
```

Output:

```
[1] 8
```

This extracts the value located at:

2nd row

3rd column

1st layer

1st group

Elements can also be extracted using functions such as:

dim() – finds dimensions of an array.

length() – finds the total number of elements.

Example:

```
dim(BR)
```

Output:

```
[1] 3 4 2 3
```

3. Replacements in R

Replacement means changing the existing value of an element with a new value. Values can be replaced by assigning a new value to a specific index position.

Syntax:

array_name[index] <- new_value

Example:

Replace an element:

```
BR[1,1,1,1] <- 100
```

SBC KARKALA

BR[1,1,1,1]
Output:
[1] 100
The original value at the specified position is replaced with 100.
Replacing Multiple Values:
Example:
BR[1:2,1,1,1] <- c(50,60)
This replaces the values in the first two rows of the first column with 50 and 60.
Applications
Subsetting, extraction, and replacement are widely used in data analysis and programming. They help in selecting required data, modifying incorrect values, updating datasets, and preparing data for statistical calculations.

Non-Numeric Values in R
In R, data values are not always numbers. R also supports **non-numeric values**, which include **characters, logical values, and factors**. These data types are used to represent text information, categories, and conditions. Non-numeric values are important for storing and analyzing real-world data such as names, labels, and classifications.

Logical Values in R
Logical values are a type of non-numeric data in R that represent the result of a condition or comparison. Logical values have only two possible states: TRUE and FALSE. They are mainly used in decision-making, filtering data, and performing comparisons in R programming.
Logical values are also called Boolean values. In R, logical values are stored using the keywords:
TRUE – represents a condition that is correct.
FALSE – represents a condition that is incorrect.
Creating Logical Values
Logical values can be created directly or generated through comparison operations.
Example 1: Creating Logical Values
x <- c(TRUE, FALSE, TRUE, FALSE)
x
Output:
[1] TRUE FALSE TRUE FALSE
Example 2: Logical Values Using Comparison Operators
a <- 10
b <- 5
a > b
Output:
[1] TRUE
Since 10 is greater than 5, the result is TRUE.
Common Logical Operators in R

Operator	Meaning	Example
>	Greater than	5 > 3 gives TRUE
<	Less than	2 < 4 gives TRUE
==	Equal to	5 == 5 gives TRUE
!=	Not equal to	5 != 3 gives TRUE
>=	Greater than or equal to	5 >= 5 gives TRUE
<=	Less than or equal to	3 <= 5 gives TRUE

Logical Operators
R also provides logical operators for combining conditions:
& (AND) – Returns TRUE only if both conditions are TRUE.

| (OR) – Returns TRUE if at least one condition is TRUE.
! (NOT) – Reverses the logical value.
Example:
x <- 10
x > 5 & x < 20
Output:
[1] TRUE
Both conditions are satisfied, so the result is TRUE.

Applications of Logical Values

Logical values are widely used in R for:
Filtering rows in datasets.
Controlling program execution using conditions.
Testing relationships between values.
Selecting specific elements from vectors and matrices.
Performing statistical analysis.

A Logical Outcome: Relational Operators in R

In R, relational operators are used to compare two values or expressions. The result of a comparison is always a logical value, which can be either TRUE or FALSE. These logical outcomes help in making decisions, filtering data, and controlling the flow of programs.
Relational operators compare values based on relationships such as greater than, less than, or equality. They are commonly used with numeric, character, and logical data.

Common Relational Operators in R

Operator	Meaning	Example	Result
>	Greater than	10 > 5	TRUE
<	Less than	3 < 8	TRUE
>=	Greater than or equal to	10 >= 10	TRUE
<=	Less than or equal to	5 <= 8	TRUE
==	Equal to	5 == 5	TRUE
!=	Not equal to	5 != 3	TRUE

Example 1: Comparing Two Values
x <- 20
y <- 10
x > y
Output:
[1] TRUE
Since 20 is greater than 10, the result is TRUE.
Example 2: Equality Comparison
a <- 5
b <- 5
a == b
Output:
[1] TRUE
The result is TRUE because both values are equal.
Example 3: Relational Operators with Vectors
R performs comparisons element by element when vectors are used.
x <- c(2, 4, 6)
x > 3

SBCKARKALA

Output:
[1] FALSE TRUE TRUE
Here, each element is compared separately with the value 3.
Importance of Relational Operators
Relational operators are useful for:
Selecting specific data values.
Filtering rows in datasets.
Performing conditional calculations.
Creating logical conditions in programs.
Comparing values during statistical analysis.

Multiple Comparisons: Logical Operators in R
Logical operators in R are used when we need to check whether multiple conditions are satisfied at the same time. They allow us to combine different logical expressions and perform operations based on the results.
Relational operators are used to compare actual values, such as numbers or characters. For example, checking whether one number is greater than another produces a logical result (TRUE or FALSE). Logical operators, on the other hand, are used to compare and combine these logical results.
Logical operators work with two types of logical values:
TRUE – represents a condition that is satisfied.
FALSE – represents a condition that is not satisfied.
R provides three main logical operations:
AND (&, &&)
OR (|, ||)
NOT (!)
AND Operator
The AND operator returns TRUE only when all conditions are TRUE. It is used when all specified conditions must be satisfied.
Example:
```
x <- 10
x > 5 & x < 20
```
Output:
[1] TRUE
Both conditions are TRUE, so the final result is TRUE.
OR Operator
The OR operator returns TRUE when at least one condition is TRUE. It is used when any one of the given conditions is sufficient.
Example:
```
x <- 10
x < 5 | x > 8
```
Output:
[1] TRUE
The second condition is TRUE, so the overall result is TRUE.
Single and Element-wise Logical Operators
The AND and OR operators have two forms:
Element-wise Operators (& and |)
Compare each element separately when working with vectors.
Used for performing multiple comparisons on data values.
Example:
```
c(TRUE, FALSE, TRUE) & c(TRUE, TRUE, FALSE)
```

SBC KARKALA

Output:
[1] TRUE FALSE FALSE
Single Comparison Operators (&& and ||)
Compare only the first element of logical expressions.
Commonly used in conditional statements such as if statements.
Example:
TRUE && FALSE
Output:
[1] FALSE
NOT Operator (!)
The NOT operator reverses a logical value.
Example:
!TRUE
Output:
[1] FALSE
Importance of Logical Operators
Logical operators are useful for:
Combining multiple conditions.
Filtering data based on requirements.
Selecting specific observations from datasets.
Controlling the execution of programs.
Performing complex data analysis.

Logicals Are Numbers in R
In R, logical values (TRUE and FALSE) can also be treated as numbers during mathematical operations. Because logical values have two possible states, they are represented internally as binary values:
TRUE = 1
FALSE = 0
When arithmetic operations are performed on logical values, R automatically converts TRUE into 1 and FALSE into 0.
Example 1: Arithmetic Operations with Logical Values
TRUE + TRUE
Output:
[1] 2
Explanation:
TRUE + TRUE
= 1 + 1
= 2
Example 2:
FALSE - TRUE
Output:
[1] -1
Explanation:
FALSE - TRUE
= 0 - 1
= -1
Example 3:
T + T + F + T + F + F + T
Output:

SBCKARKALA

[1] 4
1 + 1 + 0 + 1 + 0 + 0 + 1 = 4
Here, T represents TRUE and F represents FALSE.
Logical Values in Comparisons
Logical values can also be used with logical operators. Numeric values can sometimes be used in place of logical values:
Any non-zero number is treated as TRUE.
Zero (0) is treated as FALSE.
Example:
1 && 1
Output:
[1] TRUE
Both values are treated as TRUE because they are non-zero.
Example:
1 || 0
Output:
[1] TRUE
One value is TRUE, so the OR operation returns TRUE.
Example:
0 && 1
Output:
[1] FALSE
The first value is treated as FALSE, so the AND operation returns FALSE.

Applications of Logical Numbers

Treating logical values as numbers is useful for:
Counting the number of TRUE values.
Performing statistical calculations.
Summarizing conditions in datasets.
Converting logical results into numerical form.
For example:
sum(c(TRUE, FALSE, TRUE, TRUE))
Output:
[1] 3
The result counts the number of TRUE values.

Logical Subsetting and Extraction in R

In R, logical subsetting and extraction are techniques used to select, extract, and modify elements of vectors and other data structures using logical values (TRUE and FALSE). Instead of specifying exact index positions, we can provide a logical condition that determines which elements should be selected.
A logical vector used for selecting elements is called a logical flag vector. An element is extracted only when the corresponding value in the flag vector is TRUE. The logical flag vector should generally have the same length as the vector being accessed. If it is shorter, R automatically recycles the values to match the required length.

1. Extracting Elements Using Logical Flags

Consider the vector:
myvec <- c(5, -2.3, 4, 4, 4, 6, 8, 10, 40221, -8)
This vector contains 10 elements.
To extract the negative values, we can use their index positions:
myvec[c(2,10)]
Output:

SBC KARKALA

[1] -2.3 -8.0
The same result can be obtained using a logical flag vector:
myvec[c(FALSE, TRUE, FALSE, FALSE, FALSE,
FALSE, FALSE, FALSE, FALSE, TRUE)]
Output:
[1] -2.3 -8.0
Here, the second and tenth positions are marked as TRUE, so only those values are extracted.

2. Logical Extraction Using Conditions

Logical subsetting becomes more useful when elements need to be selected based on a condition. Instead of manually identifying indexes, we can use relational operators to create a logical vector.
For example, to find negative values:
myvec < 0
Output:
[1] FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE
This produces a logical flag vector:
TRUE → value satisfies the condition.
FALSE → value does not satisfy the condition.
This logical vector can be used directly for extraction:
myvec[myvec < 0]
Output:
[1] -2.3 -8.0
Only values less than zero are selected.

3. Recycling of Logical Flags

If the logical flag vector is shorter than the original vector, R repeats (recycles) the logical values until the required length is reached.
Example:
myvec[c(TRUE, FALSE)]
Output:
[1] 5 4 4 8 40221
Here, the pattern:
TRUE, FALSE, TRUE, FALSE, ...
is repeated across the vector, selecting every second element starting from the first.

4. Multiple Conditions Using Logical Operators

Logical operators can be combined with relational operators to perform complex selections.
Example:
myvec[(myvec > 0) & (myvec < 1000)]
Output:
[1] 5 4 4 4 6 8 10
Explanation:
myvec > 0 selects positive values.
myvec < 1000 selects values smaller than 1000.
& ensures both conditions must be TRUE.
The result contains only positive values less than 1000.

5. Replacing Values Using Logical Subsetting

Logical subsetting can also be used to modify values. Values that satisfy a condition can be replaced with a new value.
Example:
myvec[myvec < 0] <- -200
myvec

SBC KARKALA

Output:
[1] 5 -200 4 4 4 6 8 10 40221 -200
All negative values are replaced with -200.
Note:
Logical flags can be used for replacement.
Negative logical flags cannot be used to delete elements.

6. Finding Index Positions Using which()

Sometimes, we need the actual numeric positions of elements that satisfy a condition. The which() function converts a logical vector into numeric indexes.
Syntax:
which(x)
Example:
which(x = c(TRUE, FALSE, FALSE, TRUE, TRUE))
Output:
[1] 1 4 5
It returns the positions where the value is TRUE.
Example with myvec:
which(x = myvec < 0)
Output:
[1] 2 10
This shows that negative values are located at positions 2 and 10.

7. Omitting Elements Using which()

The which() function is useful when we want to remove elements based on a condition. The indexes returned by which() can be made negative to omit those positions.
Example:
myvec[-which(x = myvec < 0)]
Output:
[1] 5 4 4 4 6 8 10 40221
The negative elements are removed from the vector.
Advantages of Logical Subsetting and Extraction
Allows selection of elements without knowing their index positions.
Makes filtering large datasets easier.
Helps in modifying values based on conditions.
Reduces manual searching and improves efficiency.
Supports complex selections using logical operators.

Characters

Character strings (or simply **characters**) are a common data type in **R** used to represent **text**. A character value is written inside **single (' ')** or **double (" ")** quotation marks.

Creating a String in R

A string in R is a sequence of characters enclosed within double quotes (" ") or single quotes (' '). Strings are stored as the character data type and are used to represent text such as names, messages, and addresses.

Creating a String

To create a string, simply enter the text between quotation marks and assign it to a variable.
foo <- "This is a character string!"
foo
Output:

[1] "This is a character string!"

Length of a String

The length() function returns the number of elements in the vector. Since foo contains only one string, its length is 1.

length(foo)

Output:

[1] 1

To count the number of characters in the string, use the nchar() function.

nchar(foo)

Output:

[1] 27

Strings Can Contain Numbers

A string may also contain numeric characters, but R treats it as text rather than a numeric value.

bar <- "23.3"

bar

Output:

[1] "23.3"

Trying to perform arithmetic on a string results in an error.

bar * 2

Output:

Error in bar * 2 : non-numeric argument to binary operator

This occurs because bar is a character string, not a numeric value.

Comparing Strings

Strings can be compared using comparison operators such as == (equal to) and != (not equal to).

"alpha" == "alpha"

Output:

[1] TRUE

"alpha" != "beta"

Output:

[1] TRUE

Comparing multiple strings:

c("alpha", "beta", "gamma") == "beta"

Output:

[1] FALSE TRUE FALSE

R also supports other relational operators (<, >, <=, >=) to compare strings alphabetically.

Strings in R are character data enclosed in quotation marks. The length() function counts the number of string elements, while nchar() counts the number of characters. Strings can contain letters, numbers, and symbols, but they cannot be used directly in arithmetic operations. They can also be compared using relational and equality operators, making them useful for text processing and data analysis.

String Comparison and Special Characters in R (10 Marks Answer)

R provides several operators and functions to compare and manipulate strings. Strings are compared alphabetically (lexicographically) based on their character values.

Relational Comparison of Strings

Strings can be compared using relational operators such as <, >, <=, and >=.

Example 1:

"alpha" <= "beta"

Output:

[1] TRUE

Since "alpha" comes before "beta" alphabetically, the result is TRUE.

SBC KARKALA

Example 2:
"gamma" > "Alpha"
Output:
[1] TRUE
Here, lowercase letters are considered greater than uppercase letters in R's default character ordering.
Uppercase vs Lowercase Comparison
R distinguishes between uppercase and lowercase letters because it is case-sensitive.
Example:
"Alpha" > "alpha"
Output:
[1] TRUE
"beta" >= "bEtA"
Output:
[1] FALSE
This happens because uppercase and lowercase letters have different character codes.
Using Special Characters in Strings
Most symbols can be included in a string without any problem.
Example:
baz <- "&4 _ 3 **%.? $ymbolic non$en$e ,; "
baz
Output:
[1] "&4 _ 3 **%.? $ymbolic non$en$e ,; "
The string can contain numbers, spaces, punctuation marks, and special symbols.
Backslash (\) – Escape Character
The backslash (\) is a special character called the escape character. It is used to insert special formatting characters into a string.
Some common escape sequences are:

Escape Sequence	**Meaning**
\n	New line
\t	Tab space
\"	Double quotation mark
\\	Backslash character

Example:
cat("Hello\nWorld")
Output:
Hello
World

Concatenation
There are two main functions used to concatenate (or join together) one or more strings in R: cat() and paste(). The difference between these two functions is how their output is returned.
The cat() function sends its output directly to the console screen and does not return a value. The paste() function combines strings and returns the final character string as an R object. This returned string can be stored in a variable, passed to another function, or used for further processing.
Consider the following vector of character strings:
words <- c("awesome", "R", "is")
length(words)
Output:
[1] 3

SBCKARKALA

Displaying the vector:
words
Output:
[1] "awesome" "R" "is"
Similar to numeric and logical values, strings can also be stored in matrix or array structures.
When using cat() or paste(), the strings are passed as arguments in the order in which they should be combined.
Using cat() Function
cat(words[2], words[3], "totally", words[1], "!")
Output:
R is totally awesome !
The cat() function combines the strings and displays them directly on the screen. It does not create a character object that can be stored and reused.
Using paste() Function
paste(words[2], words[3], "totally", words[1], "!")
Output:
[1] "R is totally awesome !"
The paste() function returns a character string. The [1] in the output indicates that the result is a character vector containing one string. This output can be assigned to another variable.
Example:
sentence <- paste(words[2], words[3], "totally", words[1], "!")
sentence
Output:
[1] "R is totally awesome !"
Using the sep Argument
Both cat() and paste() functions have an optional argument called sep, which specifies the separator between strings.
Example using a separator
paste(words[2], words[3], "totally", words[1], "!", sep = "---")
Output:
[1] "R---is---totally---awesome---!"
Without any separator
paste(words[2], words[3], "totally", words[1], "!", sep = "")
Output:
[1] "Ristotallyawesome!"
If the sep argument is not specified, R uses a space as the default separator.
Example Using Empty Separator
To create a correctly spaced sentence, spaces can be manually added between strings:
cat("Do you think ", words[2], " ", words[3], " ", words[1], "?", sep = "")
Output:
Do you think R is awesome?
String concatenation is an important operation in R used to combine multiple strings. The cat() function is mainly used for displaying combined text directly on the console, whereas the paste() function creates and returns a character string that can be stored and used later. The sep argument allows users to control the spacing or symbols placed between concatenated strings.

Escape Sequences in R

An escape sequence is a combination of characters used inside a string to represent special characters or perform special formatting operations. In R, an escape sequence begins with a backslash (\), which is also called the escape character.

SBC KARKALA

Escape sequences are useful when we need to include characters that cannot be directly written inside a string, such as quotation marks, new lines, tabs, or backslashes.

Common Escape Sequences in R

Escape Sequence	Meaning	Example
\n	New line	Moves text to the next line
\t	Tab space	Inserts a tab
\"	Double quotation mark	Inserts " inside a string
\'	Single quotation mark	Inserts ' inside a string
\\	Backslash character	Inserts \ inside a string
\b	Backspace	Removes the previous character
\r	Carriage return	Moves cursor to the beginning of the line

Examples of Escape Sequences

1. New Line (\n)

The \n escape sequence is used to start a new line.

```
message <- "Hello\nWelcome to R"
cat(message)
```

Output:

```
Hello
Welcome to R
```

2. Tab Space (\t)

The \t escape sequence inserts a tab space between words.

```
text <- "Name\tAge"
cat(text)
```

Output:

```
Name    Age
```

3. Double Quote (\")

A double quote cannot be directly used inside a string enclosed by double quotes. The escape sequence \" allows it.

```
quote <- "R is called \"Statistical Computing Language\""
cat(quote)
```

Output:

```
R is called "Statistical Computing Language"
```

4. Single Quote (\')

The \' escape sequence is used to insert a single quotation mark.

```
text <- 'R language\'s features are powerful'
cat(text)
```

Output:

```
R language's features are powerful
```

5. Backslash (\\)

A double backslash is used to display a single backslash character.

```
path <- "C:\\Users\\Student\\Documents"
print(path)
```

Output:

```
[1] "C:\\Users\\Student\\Documents"
```

Importance of Escape Sequences

Escape sequences allow special characters to be included in strings.

They help format text output in a readable way.

They are useful for creating reports, messages, and file paths.

They allow quotation marks and symbols to be used inside strings.

They provide better control over string formatting.

SBC KARKALA

Escape sequences in R are special character combinations beginning with a backslash (\). They help programmers insert special characters and control the display of strings. Common escape sequences such as \n, \t, \", and \\ are widely used in text processing and output formatting.

Substrings and Matching in R

A **substring** is a part of a larger string. In R, substrings are used to extract, replace, or search for specific parts of a character string. R provides several functions for handling substrings and matching patterns, such as substr().

1. Extracting Substrings Using substr() Function in R (Detailed Explanation)

A substring is a part of a character string that is extracted from a larger string. In R programming, the substr() function is used to extract a specific portion of a string by specifying the starting position and ending position of the required characters.

The substr() function works by identifying the position of each character in a string and returning only the characters between the specified start and stop positions.

Syntax

substr(x, start, stop)

Arguments

x: The input character string from which the substring is extracted.

start: The position of the first character to extract.

stop: The position of the last character to extract.

Example 1: Extracting a Substring

```
text <- "Programming"
substr(text, start = 1, stop = 6)
```

Output

[1] "Progra"

The function extracts characters from position 1 to 6, producing:

Progra

Example 2: Extracting Characters from the Middle of a String

```
word <- "Computer"

substr(word, start = 3, stop = 6)
```

Output

[1] "mput"

Example 3: Extracting the First Few Characters of Multiple Strings

The substr() function can also work with a vector of strings.

```
names <- c("Computer", "Programming", "Language")
substr(names, 1, 4)
```

Output

[1] "Comp" "Prog" "Lang"

Here, the first four characters of each string are extracted.

2. Replacing Substrings Using substr() Function in R

The substr() function is not only used for extracting substrings but also for replacing characters or parts of a string. By assigning a new value to the substring position, the selected characters can be modified.

Syntax

substr(x, start, stop) <- value

Arguments

x: The original string.

start: Starting position of the part to replace.

stop: Ending position of the part to replace.

value: The new string that replaces the selected characters.

Example 1: Replacing a Part of a String

```
name <- "Robert"
substr(name, 1, 3) <- "Rob"
```

name
Output
[1] "Robert"
The characters from position 1 to 3 (Rob) are replaced with "Rob", so the string remains:
Robert
Example 2: Changing a Substring
text <- "I like Java"
substr(text, 8, 11) <- "R"
text
Output
[1] "I like R"
Explanation
The substring "Java" is replaced by "R".
Example 3: Replacing Multiple Characters
course <- "Data Science"
substr(course, 1, 4) <- "Big"
course
Output
[1] "Biga Science"
The characters from position 1 to 4 are replaced with the new text "Big".
Important Points About substr() Replacement
Replacement changes the original string value.
The replacement length does not need to be equal to the original substring length.
If the replacement string is shorter, remaining characters may be removed.
If the replacement string is longer, only the required portion may be used.
It is useful for modifying and cleaning text data.
The substr() function in R is an important string manipulation function. It is used for both extracting substrings and replacing parts of strings. By specifying start and stop positions, users can easily select or modify characters within a string. This function is widely used in text processing, data cleaning, and data analysis applications.

Substitution Using sub() and gsub()
In R, substitution is the process of replacing a specific part of a character string with another string. While the substr() function replaces characters based on their positions, the sub() and gsub() functions provide a more flexible method by searching for a specific pattern within a string.
The main difference between sub() and gsub() is the number of replacements they perform:
sub() replaces only the first occurrence of the matching pattern.
gsub() replaces all occurrences of the matching pattern.
1. sub() Function
The sub() function searches for a given pattern in a string and replaces only the first occurrence of that pattern with a new string.
Syntax
sub(pattern, replacement, x)
Arguments
pattern: The string or regular expression to search for.
replacement: The new string that replaces the matched pattern.
x: The input character string.
Example of sub()
text <- "R is easy. R is powerful."
sub("R", "Python", text)
Output
[1] "Python is easy. R is powerful."

The string contains "R" two times. The sub() function replaces only the first occurrence of "R" with "Python". The second "R" remains unchanged.

2. gsub() Function

The gsub() function works similarly to sub(), but it replaces every occurrence of the specified pattern in the string.

Syntax

gsub(pattern, replacement, x)

Arguments

pattern: The text or pattern to search for.

replacement: The new text used for substitution.

x: The original string.

Example of gsub()

```
text <- "R is easy. R is powerful."
gsub("R", "Python", text)
```

Output

[1] "Python is easy. Python is powerful."

Unlike sub(), the gsub() function replaces all occurrences of the matching pattern.

Difference Between sub() and gsub()

sub()	gsub()
Replaces only the first occurrence of a pattern.	Replaces all occurrences of a pattern.
Used when only one replacement is needed.	Used when multiple replacements are required.
Faster for single substitutions.	Useful for complete text replacement.

Example with Words

Using sub()

```
sentence <- "I like Java. Java is popular."
sub("Java", "R", sentence)
```

Output:

[1] "I like R. Java is popular."

Only the first "Java" is replaced.

Using gsub()

```
sentence <- "I like Java. Java is popular."
gsub("Java", "R", sentence)
```

Output:

[1] "I like R. R is popular."

Both occurrences of "Java" are replaced.

Applications of sub() and gsub()

Replacing incorrect words in text data.

Cleaning and preprocessing datasets.

Removing unwanted characters from strings.

Changing formats of dates, names, or codes.

Performing text analysis and data transformation.

The sub() and gsub() functions in R provide flexible methods for replacing text patterns inside strings. The sub() function replaces only the first matching occurrence, whereas the gsub() function replaces all matching occurrences. These functions are widely used in data cleaning, text processing, and manipulation of character data.

Factors in R

In R programming, a **factor** is a special type of data structure used to store and represent **categorical data**. Categorical data consists of values that belong to a limited number of groups or categories. Examples include gender (Male, Female), colors (Red, Blue, Green), grades (A, B, C), and customer ratings (Poor, Average, Good).

SRC KARKALA

Factors are important in R because many statistical methods require categorical variables to be stored as factors. R internally stores factor values as **integer codes** along with a set of corresponding **levels** (category names).

Identifying Categories in Factors in R

In R, a factor is used to represent categorical data. Each unique value in a factor is called a category or level. Identifying categories in a factor is important because it helps understand the groups present in the data and is useful for statistical analysis and data visualization.

Using levels() Function in R

In R, a factor is a data structure used to store categorical data. Each unique category in a factor is called a level. The levels() function is used to view, access, modify, or assign the categories (levels) of a factor.

The levels() function is one of the most important functions for working with factors because it allows users to understand what categories exist in a dataset and change category names when required.

Syntax of levels() Function

levels(x)

or

levels(x) <- value

Arguments

x: The factor whose levels need to be accessed or modified.

value: A new set of labels assigned to the factor levels.

1. Displaying Categories of a Factor

The primary use of levels() is to display all categories present in a factor.

Example

```
gender <- factor(c("Male", "Female", "Male", "Female"))
levels(gender)
```

Output

[1] "Female" "Male"

Explanation

The factor gender contains two categories:

Female

Male

The levels() function returns these unique categories.

2. Finding Levels in a Factor with Multiple Categories

```
colors <- factor(c("Red", "Blue", "Green", "Red", "Blue"))
levels(colors)
```

Output

[1] "Blue" "Green" "Red"

Explanation

Although the vector contains repeated values, R stores only the unique categories as levels.

Original data:

Red, Blue, Green, Red, Blue

Stored levels:

Blue, Green, Red

3. Changing Level Names Using levels()

The levels() function can be used to rename existing categories.

Example

```
status <- factor(c("P", "F", "P", "F"))
levels(status)
```

Output

[1] "F" "P"

SBC KARKALA

Now change the level names:
levels(status) <- c("Fail", "Pass")
status
Output
[1] Pass Fail Pass Fail
Levels: Fail Pass
The original categories:
F → Fail
P → Pass
are replaced with meaningful labels.

4. Assigning Levels While Creating a Factor

Levels can also be specified during factor creation.
Example

```
grade <- factor(
  c("A", "B", "C", "A"),
  levels = c("A", "B", "C")
)
levels(grade)
```

Output
[1] "A" "B" "C"
The levels argument defines the possible categories of the factor.

5. Reordering Levels Using levels()

The order of factor levels can be changed by assigning a new order.
Example

```
size <- factor(c("Small", "Medium", "Large"))
levels(size)
```

Output:
[1] "Large" "Medium" "Small"
The levels can be reordered:

```
levels(size) <- c("S", "M", "L")
size
```

Output:
[1] S M L
Levels: L M S

6. Using levels() with Ordered Factors

For ordered factors, levels determine the sequence of categories.
Example

```
rating <- factor(
  c("Low", "Medium", "High"),
  levels = c("Low", "Medium", "High"),
  ordered = TRUE
)
levels(rating)
```

Output
[1] "Low" "Medium" "High"
The order is:
Low < Medium < High
This allows meaningful comparisons.
7. Removing Unused Levels

SBC KARKALA

Sometimes a factor contains levels that are no longer present after filtering data.
Example
```
x <- factor(c("A", "B", "C"))
x <- x[x != "C"]
levels(x)
```
Output
[1] "A" "B" "C"
The level "C" still exists even though it is not used.
To remove unused levels:
```
x <- droplevels(x)
levels(x)
```
Output
[1] "A" "B"
Defining and Ordering Levels in Factors in R (10 Marks)
Introduction
In R, a factor is used to represent categorical data. The categories within a factor are called levels. Some factors contain categories that have no specific order, while others contain categories that follow a natural or logical order.
For example:
Unordered factor: Gender (Male, Female) – no category is higher than another.
Ordered factor: Month of birth (Jan, Feb, Mar, ...) – categories follow a fixed order.
An ordered factor helps R understand the correct sequence of categories and perform meaningful comparisons.
Creating a Character Vector
Consider a vector containing months of birth:
```
mob <- c("Apr", "Jan", "Dec", "Sep", "Nov", "Jul", "Jul", "Jun")
mob
```
Output
[1] "Apr" "Jan" "Dec" "Sep" "Nov" "Jul" "Jul" "Jun"
This vector has two problems:
1. Missing Categories
The vector does not contain all months. It contains only:
Jan, Apr, Jun, Jul, Sep, Nov, Dec
Missing months are:
Feb, Mar, May, Aug, Oct
2. Incorrect Ordering
Character vectors are arranged alphabetically, not according to calendar order.
Example:
```
mob[2]
```
Output:
[1] "Jan"
```
mob[3]
```
Output:
[1] "Dec"
Comparison:
```
mob[2] < mob[3]
```
Output:
[1] FALSE
Explanation
R compares strings alphabetically:
Dec comes before Jan

SBC KARKALA

Therefore, R gives FALSE.
However, according to the calendar:
January comes before December
So the result is not logically correct.
Defining Levels in a Factor
To solve the missing category problem, all possible categories can be defined using the levels argument of the factor() function.
First, create a vector containing all months in the correct order:

```
months <- c("Jan", "Feb", "Mar", "Apr", "May", "Jun",
            "Jul", "Aug", "Sep", "Oct", "Nov", "Dec")
```

Now create a factor with these levels:

```
mob.fac <- factor(
  x = mob,
  levels = months
)
mob.fac
```

Output

```
[1] Apr Jan Dec Sep Nov Jul Jul Jun
Levels: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
```

Explanation
Although some months are not present in the data, R recognizes all twelve months as valid categories.
Ordering Levels in a Factor
To tell R that the levels have a natural order, use:
ordered = TRUE
Example

```
mob.fac <- factor(
  x = mob,
  levels = months,
  ordered = TRUE
)
mob.fac
```

Output

```
[1] Apr Jan Dec Sep Nov Jul Jul Jun
Levels:
Jan < Feb < Mar < Apr < May < Jun < Jul < Aug < Sep < Oct < Nov < Dec
```

The < symbol indicates that the levels have a defined order.
Comparing Ordered Factors
Now R understands the correct order of months.
Example:

```
mob.fac[2]
```

Output:

```
[1] Jan
Levels: Jan < Feb < Mar < Apr < May < Jun < Jul < Aug < Sep < Oct < Nov < Dec
mob.fac[3]
```

Output:

```
[1] Dec
Levels: Jan < Feb < Mar < Apr < May < Jun < Jul < Aug < Sep < Oct < Nov < Dec
```

Comparison:

```
mob.fac[2] < mob.fac[3]
```

SBC KARKALA

Output
[1] TRUE
Explanation
R now understands:
January < December
because the factor levels are ordered according to the calendar.

LISTS AND DATA FRAMES

A list is one of the most flexible data structures in R. It is used to store a collection of different types of objects together in a single variable. Unlike vectors, matrices, and arrays, which can store only one type of data (such as all numeric or all character values), a list can contain a mixture of data types. For example, a single list can store a numeric vector, a character string, a logical vector, a matrix, a factor, or even another list. Because of this flexibility, lists are widely used in R programming for storing complex data and the output of many R functions.

Example:
foo <- list(matrix(1:4, nrow=2), c(TRUE, FALSE, TRUE), "Hello")
In this example, the list foo contains a numeric matrix, a logical vector, and a character string.
Creating a List
A list is created using the list() function. The elements to be included in the list are supplied inside the parentheses and separated by commas. Each element may be of a different data type and size. Lists are useful when you need to keep related information together without converting everything to the same data type. The order in which the elements are provided determines their position in the list.
Syntax:
list_name <- list(element1, element2, element3)
Example: foo <- list(matrix(1:4, nrow=2), c(TRUE, FALSE, TRUE), "Hello")
Here, the list stores an matrix, logical valve, and a string.
Finding the Length of a List
The length() function is used to determine the number of components or elements present in a list. It does not count the number of values inside each component; instead, it counts only the top-level elements stored in the list. This function is useful when working with large lists because it helps identify how many components are available.
Example:
length(foo)
Output:
[1] 3
This indicates that the list contains three components.

Accessing List Components Using [[ ]] (Member Reference)

The double square brackets ([[ ]]) are used to extract a single component from a list. This method is called member referencing because it retrieves one specific member of the list. Once the component is extracted, it behaves like a normal R object, and all operations applicable to that object can be performed. If the extracted component is a matrix, matrix operations can be used; if it is a character string, string functions can be applied.
Example:
foo[[1]]
This extracts the first component, which is the matrix.
Another example:
foo[[3]]
Output:
"Hello"
The extracted string can then be used in other operations.

SBCKARKALA

Performing Operations on Extracted Components

After extracting a component from a list using [[ ]], the component is treated exactly like an independent object. For example, if the extracted object is a numeric matrix, arithmetic operations can be performed on it. If it is a character string, it can be combined with other strings.

Example:

foo[[1]] + 5

This adds 5 to every element of the matrix.

Another example:

cat(foo[[3]], "Students")

Output:

Hello Students

This demonstrates that list components behave like normal R objects after extraction.

Modifying List Components

The contents of a list can be modified using the assignment operator (<-). To replace an existing component, specify its position using [[ ]] and assign a new value. This makes lists highly flexible because individual components can be updated without recreating the entire list.

Example:

foo[[3]] <- "Welcome"

The third component of the list is replaced with the new string "Welcome".

List Slicing Using [ ]

Single square brackets ([ ]) are used for list slicing. Unlike [[ ]], which returns a single object, [ ] returns a sublist containing one or more selected components. Even if only one element is selected, the result is still a list. This method is useful when multiple components need to be extracted together.

Example:

bar <- foo[c(2,3)]

This creates a new list containing the second and third components of foo.

The important difference is:

[[ ]] returns the actual object.

[ ] returns another list.

Naming List Components

List components can be given meaningful names using the names() function. Naming makes the list easier to understand and allows the components to be accessed by name rather than by position. Component names are stored as an attribute of the list.

Example:

names(foo) <- c("Matrix", "Logical", "Message")

After naming, the components appear as:

$Matrix

$Logical

$Message

Meaningful names improve readability and reduce errors when working with large lists.

Accessing Named Components Using $

Once components have been named, they can be accessed using the dollar ($) operator. This method is easier to read than numeric indexing because the programmer can directly refer to the component by its name. The $ operator works only with named lists.

Example:

foo$Matrix

This returns the matrix stored in the component named Matrix.

The same result can also be obtained by:

SBC KARKALA

foo[[1]]
Both methods return the same component.

Advantages of Lists

Lists are extremely useful because they can store multiple data types in one object. They are widely used in R programming because many built-in R functions return their results as lists. Lists can also contain other lists, making them suitable for storing complex and hierarchical data. They provide flexibility, improve code organization, and allow related objects to be grouped together.

Data Frames in R

A data frame is a two-dimensional data structure in R used for storing data in the form of rows and columns. It is one of the most commonly used structures for data analysis because it resembles a spreadsheet or database table. Each row represents one observation, while each column represents one variable. Unlike matrices, different columns in a data frame can store different types of data, such as numeric values, characters, logical values, or factors. However, all values within a single column must belong to the same data type.

Data Frames in R

A data frame **is the most commonly used data structure in** R **for storing and analyzing data. It organizes data in** rows and columns**, similar to a spreadsheet or database table.**

- **Rows** represent **records (observations)**.
- **Columns** represent **variables**.
- Each column can have a different data type (numeric, character, factor, logical, etc.).
- All columns must have the **same number of elements (equal length)**.

Creating a Data Frame

A data frame is created using the data.frame() function.

Syntax:

```
data.frame(column1, column2, ..., stringsAsFactors = FALSE)
```

Example:

```
mydata <- data.frame(
  person = c("Peter","Lois","Meg","Chris","Stewie"),
  age = c(42,40,17,14,1),
  sex = factor(c("M","F","F","M","M")),
  stringsAsFactors = FALSE
)
```

Construction of Data Frame in R

1. Creating a Data Frame

A **data frame** can be created from scratch using the data.frame() function. The data is supplied **variable by variable as vectors**. All vectors should normally have the **same length**, because each value in a vector represents one observation (row).

Syntax:

```
data.frame(variable1 = vector1,
           variable2 = vector2,
           variable3 = vector3)
```

2. Example of Data Frame Construction

```
mydata <- data.frame(
  person = c("Peter", "Lois", "Meg", "Chris", "Stewie"),
  age = c(42, 40, 17, 14, 1),
  sex = factor(c("M", "F", "F", "M", "M"))
)
```

Here, mydata is the name of the data frame. It contains three variables: **person, age, and sex**.

The person variable contains the names, age contains the ages, and sex contains the gender information. The factor() function converts the sex values into a **factor**, which is useful for representing categorical data.

3. Data Frame Output

When we print mydata, R displays it in rows and columns:

SBC KARKALA

```
  person age sex
1  Peter  42   M
2   Lois  40   F
3    Meg  17   F
4  Chris  14   M
5 Stewie   1   M
```

The data frame contains information about **five individuals**, including their first name, age, and sex.

4. Importance of Equal-Length Vectors

The vectors supplied to data.frame() should have the **same length**. This is because every position in a vector corresponds to one row or observation.

For example:

```
person = c("Peter", "Lois", "Meg")
age = c(42, 40, 17)
```

Both vectors contain three values, so they can correctly form three records.

If vectors have different lengths, R may **recycle the shorter vector** to match the longer one. This can produce incorrect or unexpected data. Therefore, it is important to use vectors of equal length when constructing a data frame.

5. Appearance of a Data Frame

A data frame is displayed in **rows and columns**, so it looks similar to a **matrix or spreadsheet**. However, unlike a matrix, different columns of a data frame can contain different types of data.

For example:

- person → character data
- age → numeric data
- sex → factor data

This spreadsheet-like structure makes data frames convenient for storing and manipulating datasets.

6. Records and Variables

Each **row** in a data frame is called a **record** or observation.

Each **column** is called a **variable**.

For the above example:

- Row 1 → Peter's record
- Row 2 → Lois's record
- person → Variable containing names
- age → Variable containing ages
- sex → Variable containing sex categories

Thus, the data frame has **5 records and 3 variables**.

1. Extracting an Individual Element

In R, an individual element of a data frame can be extracted by specifying its **row number and column number** inside square brackets [ ]. The general syntax is dataframe[row, column]. The number before the comma represents the row, and the number after the comma represents the column. For example, mydata[2,2] extracts the element present in the **second row and second column**. In the given data frame, the second row represents **Lois** and the second column represents **age**, so the result is 40. Therefore, mydata[2,2] returns the age of Lois. This method is useful when we need to access one specific value from a data frame.

```
mydata[2,2]
```

Output:

```
[1] 40
```

Here, 2 before the comma indicates the **second row**, and 2 after the comma indicates the **second column**. Thus, the individual element extracted is 40.

2. Extracting Multiple Rows from a Column

In R, multiple rows from a particular column can be extracted by specifying a **range of row numbers** and the required column number. The general syntax is dataframe[start_row:end_row, column]. The colon : operator is used to specify a sequence of row numbers. For example, mydata[3:5,3] extracts rows **3, 4, and 5** from the **third**

SECKARKALA

column. In the given data frame, the third column is sex, and rows 3, 4, and 5 correspond to **Meg, Chris, and Stewie**. Therefore, the result contains their sex values: F, M, and M. Since the sex column was created using the factor() function, the result is a **factor vector** with levels F and M.

```
mydata[3:5,3]
```

Output:

```
[1] F M M
Levels: F M
```

Here, 3:5 means rows **3 through 5**, while 3 indicates the **third column**. This technique is useful when we want to retrieve a particular variable for a selected group of observations.

3. Extracting Multiple Columns

In R, multiple columns from a data frame can be extracted by specifying their **column numbers inside the c() function**. The general syntax is dataframe[, c(column1, column2)]. When the space before the comma is left empty, R selects **all rows**. For example, mydata[,c(3,1)] extracts the **third and first columns** from all rows. The third column is sex, and the first column is person. Therefore, the resulting data frame contains the sex column first and the person column second. The order of the columns in the result is determined by the order given inside c(). This method is useful when we need to work with selected variables from a large data frame.

```
mydata[,c(3,1)]
```

Output:

```
  sex person
1   M  Peter
2   F   Lois
3   F    Meg
4   M  Chris
5   M Stewie
```

Here, c(3,1) means that the **third column is selected first**, followed by the **first column**. Since no row numbers are specified, all rows are included.

Accessing Variables and Data Frame Properties in R

1. Accessing Variables Using the $ Operator

In R, a variable (column) of a data frame can be accessed using the **dollar ($) operator**. This method is especially useful when we do not know or do not want to remember the column's index position. It is similar to accessing a named element of a list. The general syntax is dataframe$variable. For example, mydata$age accesses the age variable directly without specifying that it is the second column. This is very useful when working with **large data sets**, where remembering column numbers can be difficult.

```
mydata$age
```

Output:

```
[1] 42 40 17 14 1
```

Here, mydata is the name of the data frame, $ is the dollar operator, and age is the name of the variable being accessed.

2. Subsetting a Variable Using $

The vector returned by the $ operator can also be subsetted using square brackets [ ]. This allows us to extract a particular value from a variable. For example, mydata$age[2] first accesses the age variable and then selects its second value. Since the second person's age is 40, the result is 40. This gives the same result as mydata[2,2], but using the $ operator is often easier because we can refer to the variable by its name rather than its column number.

```
mydata$age[2]
```

Output:

```
[1] 40
```

Thus, mydata$age[2] means **the second value of the age variable**.

3. Finding the Size of a Data Frame

SBCA ARKALA

The size of a data frame can be determined using the functions **nrow()**, **ncol()**, and **dim()**. These functions help us find the number of records and variables in a data frame. This is particularly useful when working with large datasets because we can quickly determine how many rows and columns the data contains.

nrow() Function

The nrow() function returns the **number of rows** in a data frame. Each row represents a **record or observation**.

```
nrow(mydata)
```

Output:

```
[1] 5
```

This means that mydata contains **5 records**.

ncol() Function

The ncol() function returns the **number of columns** in a data frame. Each column represents a **variable**.

```
ncol(mydata)
```

Output:

```
[1] 3
```

This means that mydata contains **3 variables**.

dim() Function

The dim() function returns both the **number of rows and number of columns**.

```
dim(mydata)
```

Output:

```
[1] 5 3
```

4. Character Vectors and Factors

By default, R may convert character vectors supplied to data.frame() into **factor objects**. A factor is used to represent **categorical data**. For example, if we access the person variable using mydata$person, R may display **levels**, indicating that the variable has been converted into a factor.

```
mydata$person
```

Output:

```
[1] Peter Lois Meg Chris Stewie
Levels: Chris Lois Meg Peter Stewie
```

The important point here is the word **Levels**. Its presence indicates that person is being treated as a **factor** rather than an ordinary character vector.

5. Preventing Character-to-Factor Conversion

If we want character variables to remain as character vectors, we can use the optional argument **stringsAsFactors = FALSE** when creating the data frame. This tells R not to automatically convert character strings into factors. This is useful when variables such as names, addresses, or other text values should remain as ordinary character data.

```
mydata <- data.frame(
  person = c("Peter", "Lois", "Meg", "Chris", "Stewie"),
  age = c(42, 40, 17, 14, 1),
  sex = factor(c("M", "F", "F", "M", "M")),
  stringsAsFactors = FALSE
)
```

Now, when we access person:

```
mydata$person
```

Output:

```
[1] "Peter" "Lois" "Meg" "Chris" "Stewie"
```

The values are now displayed as **character strings**, and there are no factor levels.

SBCKARKALA

Adding Data Columns and Combining Data Frames in R

An existing data frame can be extended by adding **new records (rows)** or **new variables (columns)**. In R, the functions **rbind()** and **cbind()** can be used for this purpose. The rbind() function is used to add rows, while cbind() is used to add columns. The dollar operator $ can also be used to add a new variable to an existing data frame.

2. Adding a New Record Using rbind()

The **rbind()** function is used to add a new row or record to an existing data frame. Suppose we want to add information about a new individual named **Brian** to the mydata data frame. First, we create a new data frame containing Brian's information.

```
newrecord <- data.frame(
  person = "Brian",
  age = 7,
  sex = factor("M", levels = levels(mydata$sex))
)
```

Here, person contains Brian's name, age contains his age, and sex contains his sex. The levels() function is used to make sure that the new factor has the **same levels** as the existing sex variable.

The new record is:

```
  person age sex
1 Brian   7   M
```

Before combining data frames, it is important that the **variable names and data types match** the existing data frame. This prevents errors and maintains consistency.

Now, rbind() can be used:

```
mydata <- rbind(mydata, newrecord)
```

The resulting data frame is:

```
  person age sex
1  Peter  42   M
2   Lois  40   F
3    Meg  17   F
4  Chris  14   M
5 Stewie   1   M
6  Brian   7   M
```

Thus, rbind() increases the **number of rows** in the data frame.

3. Adding a New Variable Using cbind()

The **cbind()** function is used to add a new column or variable to an existing data frame. Suppose we want to add a new variable called **funny**, which represents the degree of funniness of each individual.

First, we create a character vector:

```
funny <- c("High", "High", "Low", "Med", "High", "Med")
```

The order of these values must correspond to the **order of records in mydata**. For example, the first value High belongs to Peter, the second High belongs to Lois, and so on.

The vector can then be converted into a factor:

```
funny <- factor(
  x = funny,
  levels = c("Low", "Med", "High")
)
```

The result is:

```
[1] High High Low Med High Med
Levels: Low Med High
```

Here, Low, Med, and High are the **levels** of the factor.

Now, the cbind() function can be used to add this variable to the data frame:

```
mydata <- cbind(mydata, funny)
```

The resulting data frame is:

```
  person age sex funny
1  Peter  42   M High
2   Lois  40   F High
```

SBCKARKALA

3 Meg 17 F Low
4 Chris 14 M Med
5 Stewie 1 M High
6 Brian 7 M Med
Thus, cbind() increases the **number of columns** in the data frame.

4. Adding a Variable Using the $ Operator
Another simple way to add a new column is by using the **dollar ($) operator**. It works similarly to adding a new member to a named list. Suppose we want to add the age of each individual in **months** instead of years. We can create a new variable called age.mon.
mydata$age.mon <- mydata$age * 12
Here, mydata$age accesses the existing age column, and multiplying it by 12 converts the age from years into months. The result is assigned to the new variable age.mon.
The resulting data frame is:
person age sex funny age.mon
1 Peter 42 M High 504
2 Lois 40 F High 480
3 Meg 17 F Low 204
4 Chris 14 M Med 168
5 Stewie 1 M High 12
6 Brian 7 M Med 84
For example, Peter's age is 42 years, so 42 × 12 = 504 months.

Logical Record Subsets in R
1. Meaning of Logical Record Subsets
Logical record subsetting is a method used in R to select only those records (rows) of a data frame that satisfy a particular condition. It is especially useful when working with large datasets because we may not want to examine every record. Instead, we can select records based on conditions such as **sex, age, marks, salary, or any other variable**. R first creates a logical vector containing TRUE or FALSE values. Records corresponding to TRUE are selected, while records corresponding to FALSE are excluded.
2. Creating a Logical Condition
Suppose we want to select all individuals who are **male**. We can compare the sex variable with "M" using the equality operator ==.
mydata$sex == "M"
Output:
[1] TRUE FALSE FALSE TRUE TRUE TRUE
Here, R checks each record in the sex column.
- TRUE → the individual is male.
- FALSE → the individual is not male.

Thus, the logical vector identifies the positions of all male records.
3. Extracting Male Records
The logical condition can be used inside the square brackets to extract only the records where the condition is TRUE.
mydata[mydata$sex == "M",]
Output:
person age sex funny age.mon
1 Peter 42 M High 504
4 Chris 14 M Med 168
5 Stewie 1 M High 12
6 Brian 7 M Med 84
The expression mydata$sex == "M" identifies the male records, while the comma after the condition indicates that **all columns** should be returned. Therefore, the result contains all variables for only the male individuals.

SRC KARKALA

4. Selecting Specific Columns Using Negative Index

After selecting the required records, we can also decide which columns should be displayed. A **negative column index** can be used to exclude a particular column. For example, column 3 is the sex column. Therefore, -3 means that the third column should be removed from the result.

```
mydata[mydata$sex == "M", -3]
```

Output:

```
  person age funny age.mon
1  Peter  42 High     504
4  Chris  14 Med      168
5 Stewie   1 High      12
6  Brian   7 Med       84
```

Here, all male records are selected, but the sex column is excluded. The negative index is useful when we know the column number that we want to remove.

5. Selecting Columns Using Variable Names

Instead of using column numbers, we can specify the **names of the variables** that we want to include. This is useful when working with large data frames because we may not know the column index positions.

```
mydata[mydata$sex == "M",
      c("person", "age", "funny", "age.mon")]
```

Output:

```
  person age funny age.mon
1  Peter  42 High     504
4  Chris  14 Med      168
5 Stewie   1 High      12
6  Brian   7 Med       84
```

Here, only the variables person, age, funny, and age.mon are returned for male individuals. The c() function combines the variable names into a character vector.

6. Using Multiple Logical Conditions

Logical conditions can be simple or complicated depending on the requirement. We can use logical operators such as **| (OR)** and **& (AND)** to combine conditions.

For example, suppose we want to select individuals who are **more than 10 years old OR have a high degree of funniness**.

```
mydata[mydata$age > 10 | mydata$funny == "High", ]
```

Output:

```
  person age sex funny age.mon
1  Peter  42   M High     504
2   Lois  40   F High     480
3    Meg  17   F Low      204
4  Chris  14   M Med      168
5 Stewie   1   M High      12
```

Here:

- mydata$age > 10 selects people older than 10.
- mydata$funny == "High" selects people with a high degree of funniness.
- | means **OR**.

Therefore, a record is selected if **either one or both conditions are TRUE**. Brian is not selected because he is 7 years old and has a medium degree of funniness.

7. When No Records Satisfy the Condition

Sometimes a condition does not match any records in the data frame. For example, suppose we want to find individuals who are **older than 45 years**.

```
mydata[mydata$age > 45, ]
```

Output:

```
[1] person age sex funny age.mon
<0 rows> (or 0-length row.names)
```

SBC KARKALA

This means that **no records satisfy the condition**. There are no individuals in mydata whose age is greater than 45.

8. Checking Whether a Subset Contains Records

The nrow() function can be used to check how many records satisfy a particular condition. If nrow() returns 0, it means that no records were found.

```
nrow(mydata[mydata$age > 45, ])
```

Output:

```
[1] 0
```

Therefore, there are **zero records** where the age is greater than 45.

If the result is greater than zero, then the subset contains one or more records.

SBC KARKALA